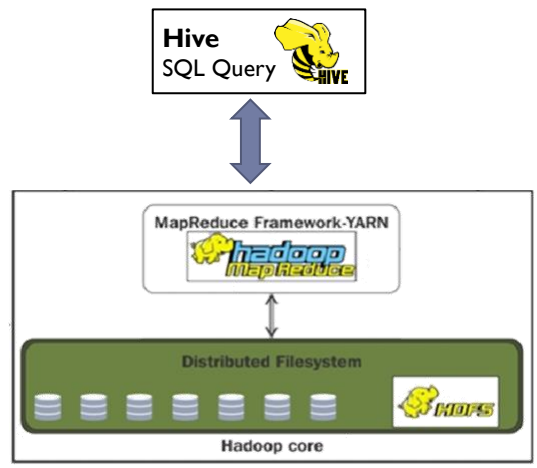
Big Data
(MEI)

João Pedro Costa
jcosta@isec.pt

1

Agenda

- Hive
- HQL



2

Hive

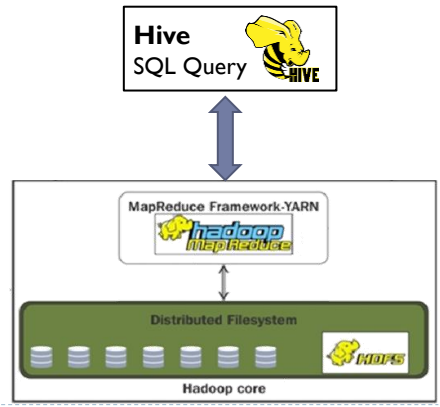
- a data warehousing infrastructure on top of **Apache Hadoop**
- facilitates reading, writing, and managing large datasets residing in distributed storage and queried using SQL syntax
- designed to maximize scalability (*scale out with more machines added dynamically to the Hadoop cluster*), performance, extensibility, fault-tolerance, and loose-coupling with its input formats
- Hadoop provides massive scale out and fault tolerance capabilities for data storage and processing on commodity hardware
- designed to enable easy data summarization, ad-hoc querying and analysis of large volumes of data.
- provides SQL which enables users to do ad-hoc querying, summarization and data analysis easily

3

Hive

What Hive Is NOT

- is not designed for online transaction processing
- it is best used for traditional data warehousing tasks



4



Hive Features

- easy access to data via SQL, thus enabling data warehousing tasks
  - as extract/transform/load (ETL)
  - reporting, and data analysis
- a mechanism to impose structure on a variety of data formats
- access to files stored either directly in HDFS or in other data storage systems such as HBase
- query execution via MapReduce, or Apache Spark
- HQL - provides standard SQL functionality, including many of SQL:2003, SQL:2011, and SQL:2016 features for analytics
- procedural language with HPL-SQL

5

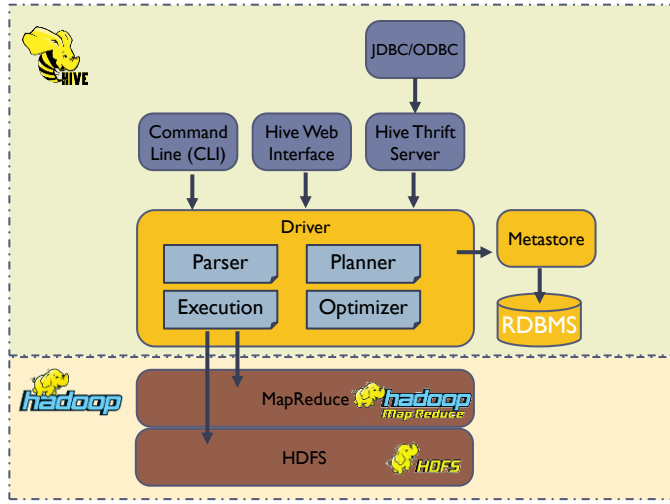


Hive

- hive compiler generates map-reduce jobs for most queries
- these jobs are then submitted to the Map-Reduce cluster
- data is accessed transparently from HDFS
- some jobs may be run locally
  - queries over small data sets, or
  - queries with multiple map-reduce jobs where the input to subsequent jobs is substantially smaller (because of reduction/filtering in the prior job)

6

Hive Architecture

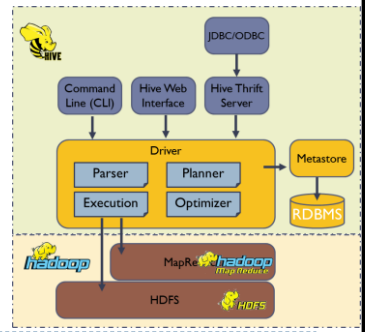


8

Hive Architecture

users submit queries through a UI

- **Driver**
  - receives the queries
  - implements the notion of session handles and provides execute and fetch APIs modeled on JDBC/ODBC interfaces
- **Compiler**
  - parses the query
  - does semantic analysis on the different query blocks and query expressions
  - eventually generates an execution plan with the help of table and partition metadata stored in metastore



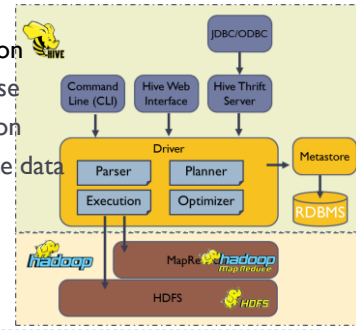
9

Hive Architecture

users submit queries through a UI

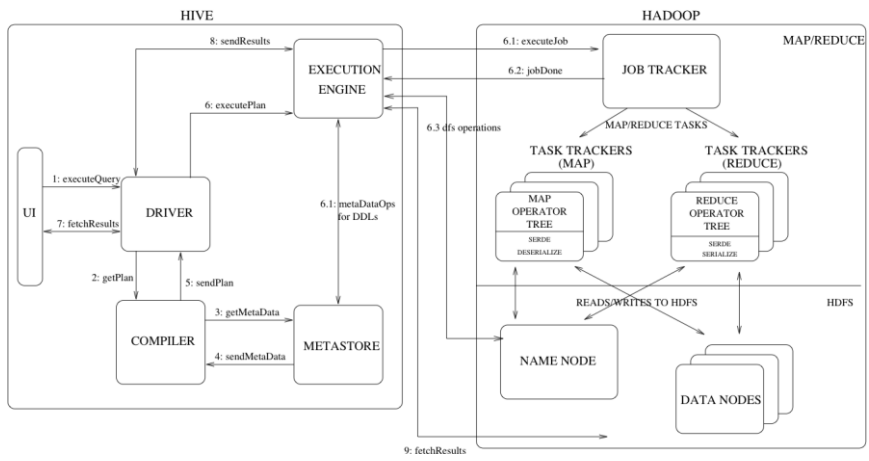
- **Execution Engine** - executes the execution plan created by the compiler
  - the plan is a DAG of stages.
  - It manages the dependencies between these different stages of the plan
  - executes these stages on the appropriate system components

- **Metastore** - stores all the structure information
  - various tables and partitions in the warehouse
  - including column and column type information
  - serializers and deserializers to read and write data
  - the HDFS files where the data is stored



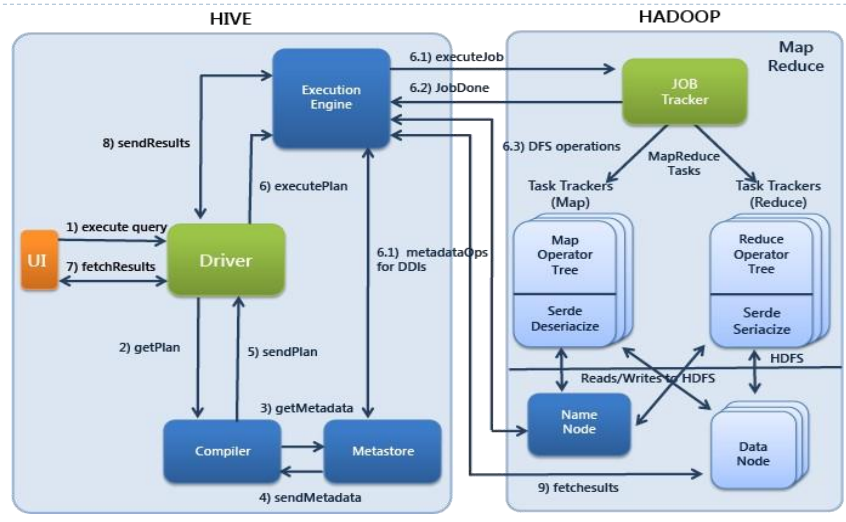
10

Hive
Components and interactions with Hadoop



11

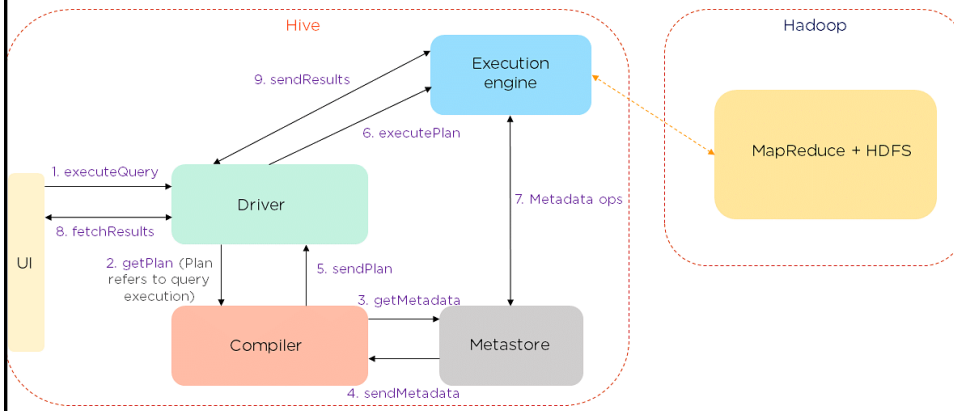
Hive
Components and interactions with Hadoop



source: https://pocfarm.wordpress.com/2016/05/09/working-of-hive/

12

Hive
Components and interactions with Hadoop



source: https://pocfarm.wordpress.com/2016/05/09/working-of-hive/

13

Hive Architecture

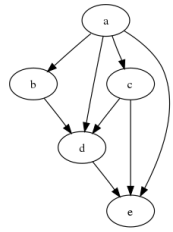
Typical query flows through the system:

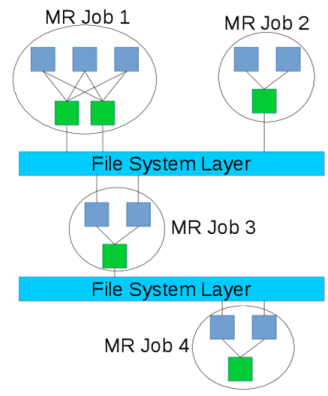
- the UI calls the execute interface to the Driver `(step 1)`
- the Driver creates a session handle for the query and sends the query to the compiler to generate an execution plan `(step 2)`
- the compiler gets the necessary metadata from the metastore `(steps 3,4)`
  - metadata is used to typecheck the expressions in the query tree as well as to prune partitions based on query predicates.
- the plan generated by the compiler `(step5)` is a DAG of stages with each stage being either a map/reduce job, a metadata operation or an operation on HDFS
  - for map/reduce stages, the plan contains map operator trees (operator trees that are executed on the mappers) and a reduce operator tree (for operations that need reducers)

Hive Architecture

DAG - directed acyclic graph

- is a directed graph with no directed cycles
- following vertices and edge directions will never form a closed loop





(a) Hive with MapReduce

source: Rahman, W. (2016). Designing and Modeling High-Performance MapReduce and DAG Execution Framework on Modern HPC Systems.



Hive Architecture

Typical query flows through the system:

- execution engine submits these stages to appropriate components `(steps 6, 6.1, 6.2, 6.3)`
  - In each task (mapper/reducer) the deserializer associated with the table, or intermediate output, is used to read the rows from HDFS and these are passed through the associated operator tree
  - the output is written to a temporary HDFS file though the serializer (happens in the mapper if the operation does not need a reduce)
  - temporary files used to provide data to subsequent map/reduce stages
- for DML the final temporary file is moved to the table's location
  - this scheme is used to ensure that dirty data is not read (file rename being an atomic operation in HDFS)
- for queries, the temporary file are read by the execution engine directly from HDFS as part of the fetch call from the Driver `(steps 7, 8, 9)`



Hive Metastore

Hive provides two important features of a data warehouse:

- **data abstraction**
  - without the data abstractions, a user has to provide information about data formats, extractors and loaders along with the query
  - this information is given during table creation and reused every time the table is referenced
  - this is very similar to the traditional warehousing systems
- **data discovery**
  - enables users to discover and explore relevant and specific data in the warehouse
  - other tools can be built using this metadata to expose and possibly enhance the information about the data and its availability



Hive Metastore Architecture

- a metadata repository that is tightly integrated with the Hive query processing system so that data and metadata are in sync
- is an object store with a database or file backed store
- database backed store which allows queriability of metadata
- since there is no clear way to implement an object store on top of HDFS due to lack of random updates to files
- Some disadvantages of using a separate data store for metadata instead of using HDFS are synchronization and scalability issues.
- can be configured to be used in :
  - **remote mode** - the metastore is a Thrift service, useful for non-Java clients
  - **embedded mode** – hive clients directly connect to an underlying metastore using JDBC. Avoids another system that needs to be maintained and monitored

18



Hive Data Model

Hive data is organized into:

- Databases
- Tables
- Partitions
- Buckets (or Clusters)

19

Hive Data Model - Databases

- namespaces function to avoid naming conflicts for tables, views, partitions, columns, and so on
- can also be used to enforce security for a user or group of users
- default database is named '`default`'

Hive Data Model - Tables

- homogeneous units of data which have the same schema
- analogous to Tables in Relational Databases
- tables can be filtered, projected, joined and unioned
- all the data of a table is stored in a directory in HDFS
- the rows are organized into typed columns similar to Relational Databases
- contains list of columns, owner, storage and SerDe information
- can also contain any user-supplied key and value data
- storage information includes location of the underlying data, file inout and output formats and bucketing information.
- SerDe metadata includes the implementation class of serializer and deserializer and any required supporting information
- this information can be provided during creation of the table

SerDe - Serializer / DeSerializer

Hive Data Model - Tables



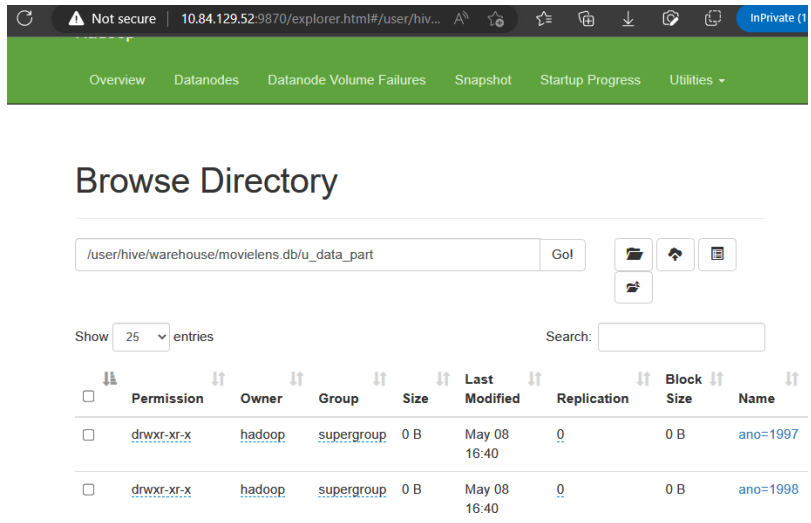
- also supports the notion of external tables wherein a table can be created on pre-existing files or directories in HDFS by providing the appropriate location to the table creation DDL
- Homogeneous units of data which have the same schema
- Eg. `page_views` table, where each row could comprise of the following columns (schema):
  - `timestamp INT` - the timestamp of when the page was viewed
  - `userid BIGINT` - identifies the user who viewed the page
  - `page_url STRING` - the location of the page
  - `referer_url STRING` - the location of the page from where the user arrived at the current page
  - `IP STRING` - the IP address from where the page request was made

Hive Data Model - Partitions



- a storage unit allows to efficiently identify rows that satisfy a specified criteria
- each can have its own columns and SerDe and storage info
- facilitates schema changes without affecting older partitions
- table can have one or more partition Keys which determines how data is stored
  - Eg. table `T` with a date partition column `ds` had files with data for a particular date stored in the `<table location>/ds=<date>` directory in HDFS
- partition columns are virtual, they are not part of the data but derived on load

Hive Data Model - Partitions



Hive Data Model - Partitions

- allows the system to prune data based on query predicates,
- eg. Partitioning `T` on attribute `ds STRING`.
  Each unique value of the partition keys defines a partition of the table
- eg. a query that gets rows from `T` that satisfy the predicate `T.ds='2008-09-01'` would only have to look at files in `<table location>/ds=2008-09-01/` directory in HDFS
- eg. all data from `'2009-12-23'` is a partition of table `T`
- for analysis on data for `2009-12-23`, the query only reads the relevant partition of the table, thereby speeding up the analysis significantly
- user must guarantee the relationship between partition name and data content
  - a partition named `2009-12-23` does not mean that it contains all or only data from that date
  - partitions are named after dates for convenience



Hive Data Model - Buckets

- **Buckets (or Clusters):**
- data in each partition may in turn be divided into Buckets based on the value of a hash function of some column
- each bucket is stored as a file in the partition directory
- E.g. the `page_views` table may be bucketed by `userid`, which is one of the columns, other than the partitions columns of the table
- bucketing allows the system to efficiently evaluate queries that depend on a sample of data
  - queries that use the SAMPLE clause on the table

Tables are not required to be partitioned or bucketed, but these abstractions allows the pruning of large quantities of data during query processing, resulting in faster query execution.



Hive Primitive Data Types

- Integers
  - `TINYINT` - 1 byte integer
  - `SMALLINT` - 2 byte integer
  - `INT` - 4 byte integer
  - `BIGINT` - 8 byte integer
- Boolean type :
  - `BOOLEAN` - TRUE/FALSE
- Floating point numbers
  - `FLOAT` - single precision
  - `DOUBLE` - Double precision
- Fixed point numbers :
  - `DECIMAL` - a fixed point value of user defined scale and precision



Hive Primitive Data Types

Primitive data type	Description	Example
TINYINT	1 byte. The postfix is Y. [-128 ;127]	10Y
SMALLINT	2 bytes. The postfix is Y. [-32,768 ; 32,767]	10S
INT	4 bytes. [-2,147,483,648 ; 2,147,483,647]	10
BIGINT	use 8 bytes. The postfix is L. [-9,223,372,036,854,775,808 ; 9,223,372,036,854,775,807]	100L
FLOAT	4-byte single precision floating point number Range [1.40129846432481707e-45; 3.40282346638528860e+38] Scientific notation is not supported. Stores very close approximations of numeric values.	1.2345679
DOUBLE	8-byte double precision floating point number. [4.94065645841246544e-324d ; 1.79769313486231570e+308d] Scientific notation is not yet supported. Stores very close approximations of numeric values.	1.2345678901234567
DECIMAL	introduced in Hive 0.11.0 with a hardcode precision of 38 digits. Hive 0.13.0 introduced user definable precision and scale. Decimal data types store exact representations of numeric values. The default definition of this type is decimal(10,0).	DECIMAL (3,2) for 3.14



Hive Primitive Data Types

- String types
  - `STRING` - characters in a specified character set
  - `VARCHAR` - characters in a specified character set with a maximum length
  - `CHAR` - characters in a specified character set with a defined length
- Date and time types
  - `DATE` - a date
  - `TIMESTAMP` - a date and time without a timezone
  - `TIMESTAMP WITH LOCAL TIME ZONE` - a point in time measured down to nanoseconds
- Binary types :
  - `BINARY` - a sequence of bytes



Hive Primitive Data Types

Primitive data type	Description	Example
BINARY	Only supports CAST to STRING and vice versa.	1011
BOOLEAN	This is a TRUE or FALSE value.	TRUE
STRING	characters with either single quotes (') or double quotes ("). C-style escaping within the strings. max size is around 2G.	'Books' or "Books"
CHAR	Most UDF will work for this type. Maximum length is 255.	'US' or "US"
VARCHAR	Most UDF will work for this type. Maximum length is 65355. If a string value being converted/assigned to a varchar value exceeds the length specified, the string is silently truncated.	'Books' or "Books"
DATE	a specific year, month, and day in the format of YYYY-MM-DD. The range of date is from 0000-01-01 to 9999-12-31.	'2013-01-01'
TIMESTAMP	a specific year, month, day, hours, minutes, seconds, and milliseconds in the format of YYYY-MM-DD HH:MM:SS[.fff...].	'2013-01-01 12:00:01.345'



Hive – Data Types

Primitive data type	Description	Example
ARRAY	list of items of the same type *[val1, val2, ...]* value accessible using *array_name[index]* for example, *fruit[0]='apple'*	['apple','orange','mango']

MAP	set of key-value pairs *{key1:val1, key2:val2, ...}*
	value accessible using *map_name[key]*
	for example, *fruit[1]="apple"*
		{
		1: "apple",
		2: "orange"
		}

STRUCT	user-defined structure of any type of fields *{val1, val2, ...}* By default, STRUCT field names will be col1, col2, and so on. value accessible using structs_name.column_name, for example, fruit.col1=1	{1, "apple"}

NAMED STRUCT	user-defined structure of any number of typed *fields {key1:val1, key2:val2, ...}*
	value accessible using *structs_name.column_name*
	for example, *fruit.apple="gala"*
		{
		"apple“ : "gala",
		"weightkg": 1
		}

UNION	a structure that has exactly any one of the specified data types. It is not commonly used.	{ 2: ["apple","orange"] }



Hive Query Language (HQL)

- an SQL-like query language for Hive.
- it mostly mimics SQL syntax for creation of tables, loading data into tables and querying the tables
- unique feature is multi-table insert
  - users can perform multiple queries on the same input data using a single HiveQL query
- allows users to embed their custom map-reduce scripts.
  - scripts can be written in any language using a simple row-based streaming interface – read rows from standard input and write out rows to standard output
  - this flexibility comes at a cost of a performance hit caused by converting rows from and to strings
  - allow users to implement scripts in the language of their choice



HQL - Compiler

P
S
L
Q

- **Parser**
  - Transform a query string to a parse tree representation
- **Semantic Analyzer**
  - transform the parse tree to an internal query representation, which is still block based and not an operator tree.
  - the column names are verified and expansions like * are performed
  - type-checking and any implicit type conversions are also performed at this stage
  - If the table under consideration is a partitioned table, all the expressions for that table are collected so that they can be later used to prune the partitions which are not needed.

HQL - Compiler



P S L Q

- **Logical Plan Generator**
  - converts the internal query representation to a logical plan, which consists of a tree of operators
  - some of the operators are relational algebra operators
    - like '`filter`','`join`',...
  - some of the operators are Hive specific and are used later on to convert this plan into a series of map-reduce jobs
  - also includes the optimizer to transform the plan to improve performance

HQL - Compiler



P S L Q

- **Query Plan Generator**
  - convert the logical plan to a series of map-reduce tasks.
  - the operator tree is recursively traversed, to be broken up into a series of map-reduce serializable tasks
  - which can be submitted later on to the map-reduce framework for HDFS
  - the reduceSink operator is the map-reduce boundary, whose descriptor contains the reduction keys
    - reduction keys in the reduceSink descriptor are used as the reduction keys in the map-reduce boundary
  - plan consists of the required samples/partitions if required by the query
  - the plan is serialized and written to a file

HQL - Optimizer



- performs more plan transformations
- the optimizer is an evolving component
  - it was rule-based and performed the following: column pruning and predicate pushdown
  - enhanced to be cost-based
    - the sorted nature of output tables can also be preserved and used later on to generate better plans
    - the query can be performed on a small sample of data to guess the data distribution, which can be used to generate a better plan

38

Hive – Client Tools



Purpose	beeline (HiveServer2)	hive (HiverServer1 Cli)
Connection	beeline –u *<jdbcurl>* -n *<username>* -p *<password>*	hive –h *<hostname>* -p *<port>*
Help	beeline –h beeline --help	hive -H
Run Query	beeline –e *"query"*	hive –e *"query"*
Run Query From File	beeline –f *<filename_with_query>*	hive –f *<filename_with_query>*

```
beeline -u jdbc:hive2://

beeline –n username -u jdbc:hive2://10.84.128.48:10000/default
```


39



Hive – Client Tool Commands

Purpose	beeline (HiveServer2)	hive (HiverServer1 Cli)
Connect	!connect *<jdbcurl>*	---
List Tables	!table	show tables;
List Columns	!column *<table_name>*	desc *<table_name>* ;
Run Query	*<HQL query>* ;	*<HQL query>* ;
Save Result Set	!record *<file_name>* !record	---
Run Shell CMD	!sh ls	! ls ;
Run dfs CMD	dfs –ls /	dfs –ls / ;
Run file of SQL	!run *<file_name>*	source *<file_name>* ;
Check Hive Version	!dbinfo	!hive --version ;
Quit	!quit	quit ;



Hive Query Language (HQL)

Databases

```
-- show databases
jdbc:hive2://> SHOW DATABASES LIKE 'my.*';
```

```
-- show databases
jdbc:hive2://> DESCRIBE DATABASE default;
```

```
+----------+-----------------------+-----------------------------------------------+
| db_name  |        comment        |                   location                    |
+----------+-----------------------+-----------------------------------------------+
| default  | Default Hive database | hdfs://10.84.129.52:9000/user/hive/warehouse  |
+----------+-----------------------+-----------------------------------------------+
```

```
-- Create a database
jdbc:hive2://> CREATE DATABASE ORDERS;
```

```
-- Use the database
jdbc:hive2://> USE movielens;
```

```
-- drop a empty database
jdbc:hive2://> DROP DATABASE IF EXISTS movielens;
```



CREATE DATABASE

Examples/Exercises:

```
CREATE DATABASE exemplo;

CREATE DATABASE exemplo _1 LOCATION '/exemplo2';

--!sh hdfs dfs -ls /user/hive/warehouse/

SHOW DATABASES;

SHOW DATABASES LIKE 'EX*';

DROP DATABASE exemplo_1;
```




Hive Query Language (HQL)

- **CREATE TABLE**

```
CREATE TABLE DEPT (
       DEPTNO int,
       DNAME  string
);
```

```
CREATE TABLE DEPT (
       DEPTNO int,
       DNAME  string
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '|'
STORED AS TEXTFILE;
```

Hive Query Language (HQL)

List tables

```
hive> show tables;
```

List tables

```
hive> show tables 'dep.*';
```


HQL - CREATE TABLE

```
CREATE [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
  ( col_name data_type [COMMENT 'col_comment'],  ... )
  [ COMMENT 'table_comment' ]
  [ PARTITIONED BY (col_name data_type [ COMMENT 'col_comment' ], ...) ]
  [ WITH SERDEPROPERTIES ('key1'='value1', 'key2'='value2', ...) ]
  [    [ROW FORMAT row_format ]
       [STORED AS file_format ]
  ]
  [ LOCATION 'hdfs_path' ]
  [ TBLPROPERTIES ('key1'='value1', 'key2'='value2', ...) ]
  [ CACHED IN 'pool_name' [ WITH REPLICATION = integer ] | UNCACHED ]
```

```
data_type:
    primitive_type
  | array_type
  | map_type
  | struct_type
```

HQL - CREATE TABLE

```
CREATE [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
  LIKE PARQUET 'hdfs_path_of_parquet_file'
  [ COMMENT 'table_comment' ]
  [ PARTITIONED BY (col_name data_type [ COMMENT 'col_comment' ], ...) ]
  [ WITH SERDEPROPERTIES ('key1'='value1', 'key2'='value2', ...) ]
  [    [ROW FORMAT row_format ]
       [STORED AS file_format ]
  ]
  [ LOCATION 'hdfs_path' ]
  [ TBLPROPERTIES ('key1'='value1', 'key2'='value2', ...) ]
  [ CACHED IN 'pool_name' [ WITH REPLICATION = integer ] | UNCACHED ]
```

```
data_type:
    primitive_type
  | array_type
  | map_type
  | struct_type
```


HQL – CREATE TABLE

Create a table `sample` with two columns and a partition column called `ds`.

```
CREATE TABLE sample (
        foo INT,
        bar STRING )
PARTITIONED BY ( ds STRING );
```

- the partition column `ds` is a virtual column. It is not part of the data but derived from the partition that a dataset is loaded into.
- By default, tables are assumed to be of text input format and the delimiters are assumed to be ^A(ctrl-a).



HQL - TABLE Storage Formats

File Type	Format	Compression	Can INSERT
TEXTFILE	Unstructured	LZO, gzip, bzip2, Snappy default file format is uncompressed text, with values separated by ASCII 0x01 characters	Yes: CREATE TABLE, INSERT, LOAD DATA, and query. If LZO compression is used, you must create the table and load data in Hive. If other kinds of compression are used, you must load data through LOAD DATA, Hive, or manually in HDFS.
PARQUET	Structured	Snappy, gzip	Yes: CREATE TABLE, INSERT, LOAD DATA, and query.
AVRO	Structured	Snappy, gzip, deflate, bzip2	No. Import data by using LOAD DATA on data files already in the right format
RCFILE	Structured	Snappy, gzip, deflate, bzip2	No. Import data by using LOAD DATA on data files already in the right format
SEQUENCEFILE	Structured	Snappy, gzip, deflate, bzip2	No. Import data by using LOAD DATA on data files already in the right format

48



HQL - CREATE TABLE ... STORED AS TEXTFILE

```
CREATE TABLE page_view_file (
     viewTime  INT,
     userid    BIGINT,
     page_url  STRING,
     ip        STRING COMMENT 'IP Address of the User')
LOCATION '/tmp/page_view_file';

CREATE TABLE page_view (
     viewTime  INT,
     userid    BIGINT,
     page_url  STRING,
     ip        STRING COMMENT 'IP Address of the User')
row format delimited fields terminated by '|'
STORED AS TEXTFILE;
```


50



HQL - CREATE TABLE ... STORED AS TEXTFILE

```
CREATE TABLE page_view_file (
     viewTime  INT,
     userid    BIGINT,
     page_url  STRING,
     ip        STRING COMMENT 'IP Address of the User')
LOCATION '/tmp/page_view_file';

CREATE TABLE page_view (
     viewTime  INT,
     userid    BIGINT,
     page_url  STRING,
     ip        STRING COMMENT 'IP Address of the User')
STORED AS TEXTFILE;
```


51



HQL - CREATE TABLE ... STORED AS TEXTFILE

```
CREATE TABLE u_data_csv (
     userid    INT,
     movieid   INT,
     rating    INT,
     unixtime  STRING)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
   "separatorChar" = "\t",
   "quoteChar"     = "'"
)
STORED AS TEXTFILE;
```


52

HQL - CREATE TABLE ... STORED AS SEQUENCEFILE




```
CREATE TABLE page_view (
     viewTime  INT,
     userid    BIGINT,
     page_url  STRING,
     ip        STRING COMMENT 'IP Address of the User')
COMMENT 'This is the page view table'
PARTITIONED BY ( dt STRING, country STRING )
STORED AS SEQUENCEFILE;
```


HQL - CREATE TABLE ... STORED AS SEQUENCEFILE




```
CREATE TABLE page_view (
     viewTime  INT,
     userid    BIGINT,
     page_url  STRING,
     ip        STRING COMMENT 'IP Address of the User')
COMMENT 'This is the page view table'
PARTITIONED BY ( dt STRING, country STRING )
CLUSTERED BY (userid) SORTED BY (viewTime) INTO 32 BUCKETS
STORED AS SEQUENCEFILE;
```



HQL - CREATE TABLE … AS SELECT …

```
CREATE TABLE T1 as
       SELECT * FROM T2;


CREATE TABLE T1
       LIKE T2;
```




HQL – Table DDL

```
DESCRIBE sample;
```

```
ALTER TABLE sample ADD COLUMNS (new_col INT);
```

```
DROP TABLE sample;
```

HQL - ALTER TABLE

```
ALTER TABLE events
     RENAME TO 3koobecaf;

ALTER TABLE pokes
     ADD COLUMNS (new_col INT);

# add comment
ALTER TABLE invites
     ADD COLUMNS (new_col2 INT COMMENT 'a comment');

# replace columns
ALTER TABLE invites
     REPLACE COLUMNS (foo INT, bar STRING,
         baz INT COMMENT 'baz replaces new_col2');
```


HQL - ALTER TABLE

```
# drop table
DROP TABLE retail;

# drop partition
ALTER TABLE retail
     DROP PARTITION (ds ='2008');
```

Note that any data for this table or partitions will be dropped and may not be recoverable.

HQL - LOAD DATA from Filesystem



```
# copy and load from filesystem
LOAD DATA LOCAL INPATH './examples/files/kv1.txt'
          INTO TABLE invites;

# copy and load from filesystem and overwrite
LOAD DATA LOCAL INPATH './examples/files/kv1.txt'
OVERWRITE INTO TABLE invites;
```


61

HQL - LOAD DATA from HDFS



```
# “copy” (MOVE) from hdfs
LOAD DATA INPATH '/input/kv1.txt'
          INTO TABLE invites;

# “copy” (MOVE) load from hdfs and overwrite
LOAD DATA INPATH '/input/kv1.txt'
OVERWRITE INTO TABLE invites;
```


62

HQL - LOAD DATA into partitions



```
# “copy” (MOVE) from hdfs
LOAD DATA INPATH '/input/kv1.txt'
OVERWRITE INTO TABLE invites
PARTITION (ds='2008-08-15');


# “copy” (MOVE) from hdfs
LOAD DATA INPATH '/input/kv2.txt'
OVERWRITE INTO TABLE invites
PARTITION (ds='2010-01-15');
```


HQL - INSERT



```
#
INSERT OVERWRITE TABLE invites1
    SELECT * FROM invites;

INSERT OVERWRITE DIRECTORY '/tmp/reg_4'
    SELECT * FROM invites;

INSERT OVERWRITE LOCAL DIRECTORY '/tmp/rl'
    SELECT * FROM invites;
```

HQL - INSERT

```
INSERT OVERWRITE TABLE emp1
    SELECT a.bar, count(*)
    FROM   emp a
    WHERE  a.foo > 0
    GROUP BY a.bar;

FROM emp a
INSERT OVERWRITE TABLE emp1
    SELECT a.bar, count(*)
    WHERE  a.foo > 0
    GROUP BY a.bar;
```


HQL - MultiTable Insert

to load data into all country partitions in a particular day, you have to add an insert statement for each country in the input data.

```
FROM page_view_stg pvs
INSERT OVERWRITE TABLE page_view
   PARTITION (dt='2010-05-10', country='US')
   SELECT pvs.viewTime, ... WHERE pvs.country = 'US'
INSERT OVERWRITE TABLE page_view
   PARTITION (dt='2010-05-10', country='CA')
   SELECT pvs.viewTime, ... WHERE pvs.country = 'CA'
INSERT OVERWRITE TABLE page_view
   PARTITION (dt='2010-05-10', country='UK')
   SELECT pvs.viewTime, ... WHERE pvs.country = 'UK'
```

requires:

- prior knowledge of the list of countries that exists in the input data
- the creation of the partitions beforehand



HQL - MultiTable Insert

- is inconvenient
  - if the list changes (e.g another day), you have to modify the insert DML as well as the partition creation DDLs.
- is inefficient
  - each insert statement may be turned into a MapReduce Job

Dynamic-partition insert (or multi-partition insert)

- dynamically determines which partitions should be created and populated while scanning the input table.

```
FROM page_view_stg pvs
INSERT OVERWRITE TABLE page_view PARTITION(dt='2010-05-10', country)
SELECT pvs.viewTime, ...
```



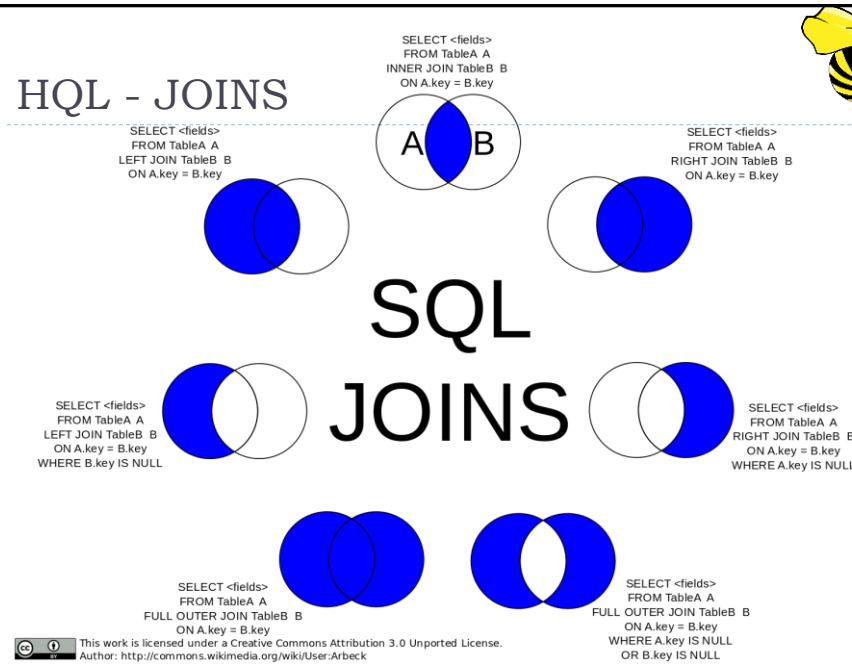

HQL - MultiTable Insert

```
FROM src
  INSERT OVERWRITE TABLE dest1
    SELECT src.*
    WHERE src.key < 100
  INSERT OVERWRITE TABLE dest2
    SELECT src.key, src.value
    WHERE src.key >= 100 and src.key < 200
  INSERT OVERWRITE TABLE dest3
  PARTITION ( ds='2008-04-08', hr='12‘ )
    SELECT src.key
    WHERE src.key >= 200 and src.key < 300
  INSERT OVERWRITE LOCAL DIRECTORY '/tmp/dest4.out'
    SELECT src.value
    WHERE src.key >= 300;
```

HQL - SELECT

```
[WITH name AS (select_expression) [, ...] ]
SELECT
  [ALL | DISTINCT]
  [STRAIGHT_JOIN]
  expression [, expression ...]
FROM table_reference [, table_reference ...]
[[FULL | [LEFT | RIGHT] INNER | [LEFT | RIGHT] OUTER | [LEFT | RIGHT] SEMI |
[LEFT | RIGHT] ANTI | CROSS]
  JOIN table_reference
  [ON join_equality_clauses | USING (col1[, col2 ...]] ...
WHERE conditions
GROUP BY { column | expression [, ...] }
HAVING conditions
ORDER BY { column | expression [ASC | DESC] [NULLS FIRST | NULLS LAST]
[, ...] }
LIMIT expression [OFFSET expression]
[UNION [ALL] select_statement ] ...]
```


HQL - JOINS





HQL - Joins

Join

```
SELECT * FROM customer c
JOIN order_cust o ON ( c.id=o.cus_id );
```

Using:

the customer file is a slash delimited file

```
CREATE TABLE customer (
        id      INT,
        name    STRING,
        address STRING )
ROW FORMAT DELIMITED FIELDS TERMINATED BY '#';
```

the orders file is a tab delimited file

```
CREATE TABLE order_cust (
        id      INT,
        cus_id  INT,
        prod_id INT,
        price   INT )
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```




HQL - Joins

Select all columns from partition `ds='2012-02-24'` of table `sample`.

- the results displayed on the console

```
SELECT * FROM sample WHERE ds='2012-02-24';
```

- the results stored in files into a HDFS directory.

```
INSERT OVERWRITE DIRECTORY '/tmp/hdfs_out'
SELECT * FROM sample WHERE ds='2012-02-24';
```

- the results stored in files into a local directory.

```
INSERT OVERWRITE LOCAL DIRECTORY '/tmp/sample-out'
SELECT * FROM sample;
```

NOTE: The number of files depends on the number of mappers)

NOTE: partition columns if any are selected by the use of *.
They can also be specified in the projection clauses.

HQL - Joins



```
SELECT *
FROM customer c
JOIN order_cust o ON (c.id=o.cus_id);
```

- The query joins two different tables.
- It needs just one statement to do the join operation.
- To do this using hadoop, the code would be pretty complex.
- Does not matter the format of each file.
  - there is no need to write different mapper for each file
  - it is managed by hive.

HQL - Joins



- Queries with sub-queries is also supported in the from clause

```
SELECT c.id,c.name,c.address,ce.exp
FROM customer c JOIN (
    SELECT cus_id,sum(price) AS exp
    FROM order_cust
    GROUP BY cus_id
  ) ce
ON (c.id=ce.cus_id);
```

HQL - SELECT



SELECTS and FILTERS

```
SELECT * FROM sample WHERE ds='2012-02-24';
```

select all columns from partition `ds='2012-02-24'` of table `sample`.

the results are displayed on the console and not stored anywhere

```
INSERT OVERWRITE DIRECTORY '/tmp/hdfs_out'
SELECT * FROM sample WHERE ds='2012-02-24';
```

select all columns from partition `ds='2012-02-24'` of table `sample` into an HDFS directory.

result data is in files (depending on the number of mappers) in that directory.

```
INSERT OVERWRITE LOCAL DIRECTORY '/tmp/hive-sample-out'
SELECT * FROM sample;
```

selects all columns from table `sample` and store the result into a local directory

HQL – Aggregations and GROUP BY



- select the max value of foo.

```
SELECT MAX(foo) FROM sample;
```

- Select that groups the ds, sums the foo values for a given ds and count the number of rows for the given ds.

```
SELECT ds, COUNT(*), SUM(foo)
FROM sample
GROUP BY ds;
```

- Select and insert the output into a table.

```
FROM sample s
INSERT OVERWRITE TABLE bar
SELECT s.bar, count(*) WHERE s.foo > 0
GROUP BY s.bar;
```

HQL - CREATE VIEW

```
CREATE VIEW [IF NOT EXISTS] view_name
    [ (column_name [COMMENT 'column_comment']
      [, ...]) ]
    [ COMMENT 'view_comment' ]
  AS select_statement
```


HQL - FUNCTIONs

- Numerical functions
- Date time functions
- String functions
- UDF functions
- MR scripts

HQL - User-Defined Function (UDF)



- extend UDF to create a new UDF classes
- UDF classes must implement one, or more, **`evaluate`** methods, which will be called by Hive.
- **`"evaluate"`** must return a value (it may be **`"null"`**)

```
package com.example.hive.udf;

import org.apache.hadoop.hive.ql.exec.UDF;
import org.apache.hadoop.io.Text;

public final class Lower extends UDF {
  public Text evaluate(final Text s) {
    if (s == null) { return null; }
    return new Text(s.toString().toLowerCase());
 }
}
```


HQL - User-Defined Function (UDF)



- Compile and include in the hive classpath
- After hive is started up, you have to register the function.
- To register a class

```
CREATE FUNCTION my_lower
AS 'com.example.hive.udf.Lower';
```

- Afterwards, the function can be used in some sentence.

```
SELECT my_lower(title), sum(freq)
FROM titles
GROUP BY my_lower(title);
```

HQL – MapReduce Scripts



- can also plug in their own custom mappers and reducers in the data stream
- To run a custom mapper script and a custom reducer script the user can issue the following command which uses the TRANSFORM clause to embed the mapper and the reducer scripts

```
SELECT TRANSFORM (foo, bar)
USING 'python ./my_append.py'
FROM sample;
```


HQL - User-Defined Function (UDF)



my_append.py

```
for line in sys.stdin:
    line = line.strip()
    key = line.split('\t')[0]
    value = line.split('\t')[1]
    print (key+str(i)+'\t'+value+str(i))
    i=i+1
```

- By default, columns will be transformed to STRING and delimited by TAB before feeding to the user script;
- similarly, all NULL values will be converted to the literal string \n in order to differentiate NULL values from empty strings.
- The standard output of the user script will be treated as TAB-separated STRING columns, any cell containing only \n will be re-interpreted as a NULL, and then the resulting STRING column will be cast to the data type specified in the table declaration in the usual way.

HQL - FUNCTIONs



abs(numeric_type a)

Returns the absolute value. Return type: Same as the input value

ceil(double a), ceil(decimal(p,s) a), ceiling(double a), ceiling(decimal(p,s) a), dceil(double a), dceil(decimal(p,s) a)

Returns the smallest integer that is greater than or equal to the argument. Return type: bigint or decimal(p,s) depending on the type of the input value

floor(double a), floor(decimal(p,s) a), dfloor(double a), dfloor(decimal(p,s) a)

Returns the largest integer that is less than or equal to the argument

Return type: bigint or decimal(p,s) depending on the type of the input

mod(double a, double b), mod(float a, float b)

Returns the modulus of a floating-point number. Return type:float or double

fmod(double a, double b), fmod(float a, float b)

Returns the modulus of a floating-point number. Return type:float or double

84

HQL - FUNCTIONs



bin(bigint a)

Returns the binary representation of an integer value, that is, a string of 0 and 1 digits. Return type: string

hex(bigint a)

Returns the hexa representation of an integer value, that is, a string of 0 and 1 digits. Return type: string

unhex(string a)

Returns a string of characters with ASCII values corresponding to pairs of hexadecimal digits in the argument.Return type: string

conv(bigint n, int from_base, int to_base)

conv(string s, int from_base, int to_base)

Returns a string representation of the first argument converted from from_base to to_base. The first argument can be specified as a number or a string. For example, conv(100, 2, 10) and conv('100', 2, 10) both return '4'.

Return type: string

85

HQL - FUNCTIONs

rand()

rand(bigint seed)

Returns a random value between 0 and 1. After rand() is called with a seed argument, it produces a consistent random sequence based on the seed value. Return type: double

round(double a), round(double a, int d), round(decimal a, int_type d)

Rounds a floating-point value. By default (with a single argument), rounds to the nearest integer.

trunc(double a), trunc(double a, int d)

trunc(decimal a, int_type d)

Rounds a floating-point value. By default (with a single argument), rounds to the nearest integer.

86

HQL - DATE FUNCTIONS

add_months(timestamp date, int months), add_months(timestamp date, bigint months)

Returns the specified date and time plus some number of months. Returns a date

date_add(timestamp startdate, int days)

Adds a specified number of days to a TIMESTAMP value. Return type: date

date_sub(timestamp startdate, int days)

substracts a specified number of days to a TIMESTAMP value. Return type: date

day(timestamp date)

Returns the day field from the date portion of a TIMESTAMP. The value represents the day of the month, in the range 1-31, or less for months without 31 days. Return type: int

dayofweek(timestamp date)

Returns the day field from the date portion of a TIMESTAMP, corresponding to the day of the week. The range of return values is 1 (Sunday) to 7 (Saturday). Return type: int

extract(unit FROM timestamp)

Returns one of the numeric date or time fields from a TIMESTAMP value. Unit argument: The unit string can be one of year, month, day, hour, minute, second, or millisecond. This argument value is case-insensitive.

87

HQL - DATE FUNCTIONS



current_timestamp()

Alias for the now() function. Return type: timestamp

from_unixtime(bigint unixtime[, string format])

Converts the number of seconds from the Unix epoch to the specified time into a string in the local time zone. Return type: string (like TO_CHAR)

unix_timestamp()

unix_timestamp(string datetime), unix_timestamp(string datetime, string format), unix_timestamp(timestamp datetime)

Returns a Unix time, which is a number of seconds elapsed since '1970-01-01 00:00:00' UTC. If called with no argument, the current date and time is converted to its Unix time. If called with arguments, the first argument represented as the TIMESTAMP or STRING is converted to its Unix time. Return a bigint

months_between(timestamp newer, timestamp older)

Returns the number of months between the date portions of two TIMESTAMP values, as an INT representing only the full months that passed. Return a int

88

HQL - STRING FUNCTIONS



length(string a), char_length(string a)

Returns the length in characters of the argument string.

find_in_set(string str, string strList)

Returns the position (starting from 1) of the first occurrence of a specified string within a comma-separated string. Returns NULL if either argument is NULL, 0 if the search string is not found, or 0 if the search string contains a comma. Return type: int

instr(string str, string substr)

Returns the position (starting from 1) of the first occurrence of a substring within a longer string. Return type: int

locate(string substr, string str[, int pos])

Returns the position (starting from 1) of the first occurrence of a substring within a longer string, optionally after a particular position. Returns an int

repeat(string str, int n)

Returns the argument string repeated a n number of times. Returns s string

reverse(string a)

Returns the argument string with characters in reversed order

89

HQL - STRING FUNCTIONS



parse_url(string urlString, string partToExtract [, string keyToExtract])

Returns the portion of a URL corresponding to a specified part.

Part argument can be 'PROTOCOL', 'HOST', 'PATH', 'REF', 'AUTHORITY', 'FILE', 'USERINFO', or 'QUERY'. Uppercase is required for these literal values.

When requesting the QUERY portion of the URL, you can optionally specify a key to retrieve just the associated value from the key-value pairs in the query string. Return type: string

split_part(string source, string delimiter)

Returns the nth field within a delimited string.

The fields are numbered starting from 1.

The delimiter can consist of multiple characters, not just a single character.

All matching of the delimiter is done exactly, not using any regular expression patterns. Return type: string